

TECHNICAL ANALYSIS DOCUMENT

IT Service Management System

Architecture, security, data and workflow design

Author	Ukbe Taha Şahinkaya
Institution	Sakarya University, Software Engineering
Document Type	Technical Analysis Document

Contents

1. Introduction	4
1.1. Purpose and Scope	4
1.2. Architectural Design Goals	4
2. System Architecture	5
2.1. Architectural Style	5
2.2. Container Diagram and Routing	5
2.3. Components and Responsibilities	6
3. Technology Stack	7
4. Security and Identity Architecture	8
4.1. Keycloak and OpenLDAP Integration	8
4.2. Authentication Flow	8
4.3. Role-Based Access Control (RBAC)	8
4.4. Internal Endpoints and Rate Limiting	10
5. Business Processes and jBPM Workflow	11
5.1. Ticket State Machine	11
5.2. BPMN Process Model and Signals	12
5.3. SLA Mechanism	12
5.4. Resilience: Circuit Breaker and Callback	13
6. Database Design and Data Dictionary	14
6.1. Approach and Standards	14
6.2. Entity-Relationship Diagram	15
6.3. Table Inventory	15
6.4. Data Dictionary (Selected Tables)	16
7. Backend Architecture (REST API)	18
7.1. Layered Structure	18
7.2. Error Handling and API Contract	18
7.3. Critical API Endpoints	18
8. Frontend and Mobile Clients	20
9. Observability	21
10. Resilience and Performance	22
11. Testing and Quality Assurance	23
12. Deployment and CI/CD	24
13. Conclusion and Assessment	25

List of Figures and Tables

Figure 1. Container architecture and traffic flow	5
Figure 2. Authentication and authorization flow	8
Figure 3. Additive role model and permission axes	9
Figure 4. Ticket state machine	11
Figure 5. jBPM SLA process flow	12
Figure 6. SLA pause / resume timeline	13
Figure 7. Entity-relationship (ER) diagram	15
Figure 8. Layered backend architecture	18
Figure 9. Observability data pipeline	21
Figure 10. CI/CD pipeline	24

1. Introduction

1.1. Purpose and Scope

This document was prepared to lay out, in detail, the current technical architecture, design decisions and system requirements of the IT Service & Support Ticket Management System. The system is built using modern software engineering disciplines as a scalable, secure and observable enterprise support platform.

The document covers the system architecture, the technology stack, the security and identity model, the workflow (jBPM) integration, the database design, the backend and frontend architectures, the observability infrastructure, the testing strategy and the deployment approach. A separate *Analysis Document* has been prepared for evaluating the system from a business and user perspective.

1.2. Architectural Design Goals

The architecture has been designed around the following core goals:

- **Clear separation of concerns:** API, artificial intelligence, identity, workflow, data and observability each run as independent, separately deployable services.
- **Stateless and horizontally scalable application services:** Session state is not held on the server; all state is stored in PostgreSQL and Redis.
- **Externalized configuration:** 12-factor principles: environment variables, Spring profiles and `application.yml`.
- **End-to-end observability:** Generation of logs, traces and metrics; all converging in a single backend.
- **Graceful degradation:** The core API keeps running even if the workflow or AI service is interrupted.

Polyglot Monorepo

The system is a containerized, polyglot structure in which five buildable components (backend, IIm-service, web frontend, mobile app, workflow kjar) and the shared infrastructure are orchestrated with `docker-compose.yml` within a single repository (monorepo).

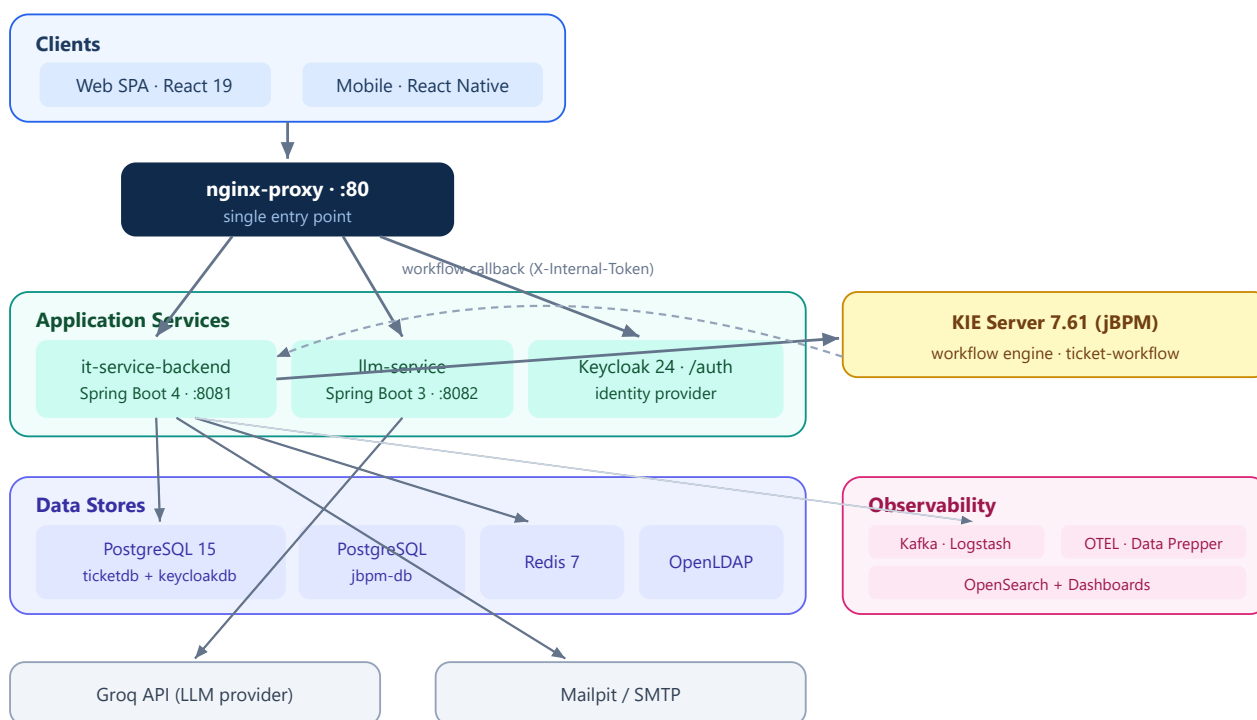
2. System Architecture

2.1. Architectural Style

Aspect	Approach
Topology	Microservice-leaning: discrete containers behind a single reverse proxy (nginx)
Backend	Classic layered architecture (Controller → Service → Repository)
Frontend	Single-Page Application (SPA, React) + React Native mobile client
Communication	Synchronous REST (JSON); STOMP/WebSocket for live updates; HTTP callbacks from the workflow engine
Identity	Externalized to Keycloak, so applications never store passwords
State	Application services are stateless (JWT-based); all state lives in PostgreSQL / Redis
Configuration	12-factor: environment variables, Spring profiles, <code>application.yml</code>
Schema management	Versioned database migrations (Flyway); Hibernate in <code>validate</code> mode

2.2. Container Diagram and Routing

All external traffic enters through the **nginx** reverse proxy on port `80`. In a near-production deployment, no application service or data store is exposed directly to the outside.



Arrows show the principal data flow; each component communicates only on the internal bridge network.

Figure 1. Container architecture and traffic flow

The nginx reverse proxy routes incoming requests to the backend services by path:

Path	Target Service
/	it-service-frontend (static SPA)
/api/v1/	it-service-backend:8081
/api/v1/ai/	llm-service:8082
/auth/	keycloak-iam:8080

2.3. Components and Responsibilities

Component	Stack	Responsibility
it-service-backend	Spring Boot 4 / Java 21	Core REST API for tickets, SLA, users, comments, attachments, notifications and dashboards. Owner of the <code>ticketdb</code> schema.
llm-service	Spring Boot 3 / Java 21	AI-assisted ticket summarization via the Groq API. Shares <code>ticketdb</code> with an isolated Flyway history table.
it-service-frontend	React 19 + Vite	Web SPA with role-scoped interfaces; navigation is composed from the union of the user's roles.
it-service-mobile	React Native + Expo	Mobile client with functional parity to the web app.
ticket-workflow-kjar	jBPM / BPMN 2.0	The <code>ticket-lifecycle</code> process definition deployed to the KIE Server.
Keycloak	Keycloak 24	Identity provider: OAuth2/OIDC, <code>TicketSystemRealm</code> , users federated from LDAP.
OpenLDAP	OpenLDAP	Directory server, the source of truth for user accounts.
KIE Server	jBPM 7.61 (WildFly)	Hosts the workflow process; backed by a separate PostgreSQL.
PostgreSQL	PostgreSQL 15	<code>ticketdb</code> (application) and <code>keycloakdb</code> ; <code>jbpm-db</code> is a separate instance.
Redis	Redis 7	Distributed rate-limit buckets (Bucket4j); groundwork for future cache/queue use.
Kafka + Logstash	Kafka 3.7	Buffer and consumer that ship logs to OpenSearch.
OTEL Collector + Data Prepper	OpenTelemetry	Telemetry ingestion; fans out traces/logs/metrics to OpenSearch.
OpenSearch	OpenSearch 3.6	Unified store with Dashboards for logs, traces and metrics.
nginx	nginx	Reverse proxy and single entry point.
data-generator	Java	Standalone tool that generates realistic demo data through the API.

3. Technology Stack

The tables below list the technologies actually used in the project and their versions, grouped by layer.

Backend

Language / Runtime	Java 21
Framework	Spring Boot 4.0.5
Security	Spring Security · OAuth2 Resource Server
Data access	Spring Data JPA · Hibernate
Migration	Flyway (V1–V38)
Cache	Caffeine
Rate limiting	Bucket4j (Lettuce/Redis)
Workflow client	kie-server-client 7.61
Resilience	Resilience4j (circuit breaker)
Telemetry	Micrometer Tracing · OpenTelemetry
Logging	Log4j2 (Logback excluded)
API docs	springdoc OpenAPI (Swagger UI)

Mobile

Framework	React Native 0.81 · Expo 54
Navigation	React Navigation 7
Identity	expo-auth-session (OIDC PKCE)
Secure storage	expo-secure-store
Real-time	@stomp/stompjs 7
Internationalization	i18next · react-i18next

Frontend (Web)

Library	React 19.2
Bundler	Vite 8
Styling	Tailwind CSS 4
Routing	react-router-dom 7
HTTP	axios
Identity	keycloak-js 26
Real-time	@stomp/stompjs 7 (WebSocket)
Charts	Recharts 3
Icons	lucide-react
Internationalization	i18next · react-i18next
Testing	Vitest · Testing Library · jsdom

Data, Infrastructure & Observability

Database	PostgreSQL 15 (ticketdb, keycloakdb, jbpmbpm-db)
Cache/Queue	Redis 7
Identity & Directory	Keycloak 24 · OpenLDAP
Workflow	jBPM 7.61 / KIE Server (WildFly)
Messaging	Kafka 3.7 · Logstash
Telemetry	OTEL Collector · Data Prepper
Search/Dashboard	OpenSearch 3.6 + Dashboards
Containerization	Docker Compose · Kubernetes (kind/kustomize)
CI/CD	GitHub Actions · Docker Hub
Quality	JaCoCo · SonarQube · Testcontainers

4. Security and Identity Architecture

The security infrastructure is built on the **stateless** architecture considered standard in modern web applications. Sessions are not held on the server; security is enforced entirely on a JWT (JSON Web Token) basis. Identity and authorization are provided through the integration of **Keycloak**, a central IAM tool, with **OpenLDAP**, a directory server.

4.1. Keycloak and OpenLDAP Integration

User identities are managed centrally and synchronized with external systems. To this end, **User Federation** has been set up between Keycloak and OpenLDAP:

- **OpenLDAP**: The source of truth holding users' basic identity information (username, email, first name, last name). LDAP groups are mapped to Keycloak realm roles.
- **Keycloak (TicketSystemRealm)**: Receives login requests, delegates password verification to LDAP, and on a successful login issues the JWT (signed with RS256) that serves as a digital identity.

4.2. Authentication Flow

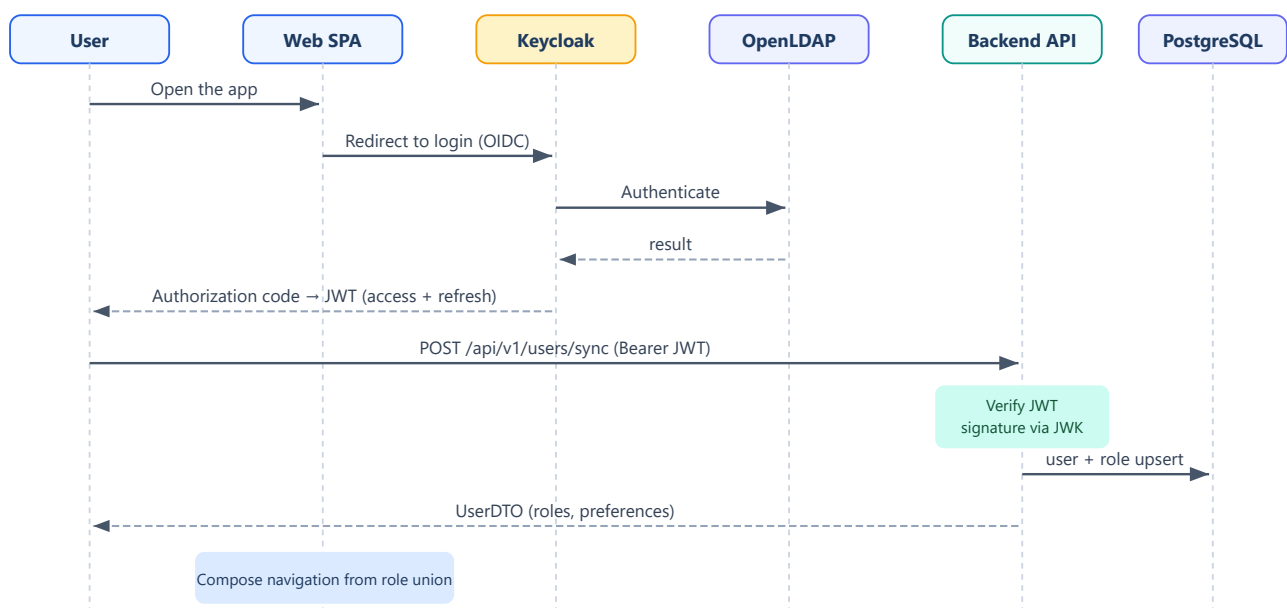


Figure 2. Authentication and authorization flow

The backend is a pure **OAuth2 Resource Server**: stateless, it accepts only JWTs and validates the signature on the fly against the Keycloak realm's JWK set. There is no server-side session; CSRF is disabled (no cookies are used).

4.3. Role-Based Access Control (RBAC)

Keycloak sends roles inside the JWT under `realm_access.roles`. A custom `JwtAuthenticationConverter` turns these roles into `ROLE_*` authorities. Endpoint security is enforced with method-level `@PreAuthorize` annotations. Scope/claim decisions in the service layer go through the `util/AuthRoles` helpers.

The authorization model is **additive**: a user carries a set of roles and their effective rights are the union of those roles. The roles span three axes. The sole constraint is that `customer` is a **singleton** role:

`UserService` validation and the role-selection modals reject combining `customer` with any staff role, keeping the end-user and staff identities disjoint.

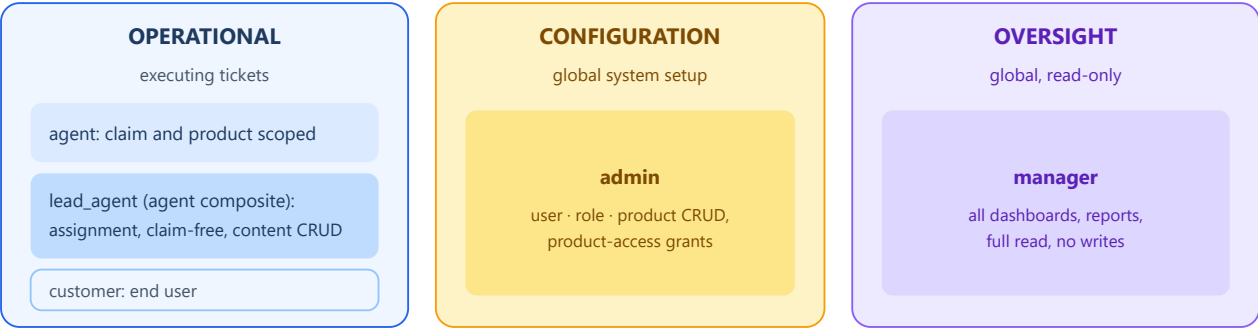


Figure 3. Additive role model and permission axes

The permission matrix below summarizes each role's access to the critical modules:

Role	Ticket Management	Comment	Configuration / Report
customer	Sees and creates only their own tickets	EXTERNAL only	Personal self-scoped Overview dashboard; completes CSAT
agent	Sees the authorized product pool, claims, changes status	EXTERNAL + INTERNAL	Personal My-Performance dashboard; logs effort, receives SLA warnings
lead_agent	Assigns, acts without a claim; manages product content	EXTERNAL + INTERNAL	Product-scoped team & per-product dashboards; drills into agents
admin	Administrative operations including ticket deletion	All	User/role/product CRUD + product-access grants; opens any user's dashboard
manager	Sees all tickets (read-only)	Sees all	All system, per-product & per-user dashboards and KPI reports

Role credentials are held in Keycloak, cached in the `user_roles` table (Flyway V37) and synchronized via `/users/sync`. The old `users.role` column is retained only as the "primary/display role"; real authorization is done with JWT authorities + `@PreAuthorize`.

4.4. Internal Endpoints and Rate Limiting

Topic	Implementation
Internal endpoints	<code>/api/v1/internal/**</code> bypasses the JWT; it is protected by a shared <code>X-Internal-Token</code> header (used only by the KIE Server callback). The comparison is constant-time against timing attacks.
Anonymous permit-list	Authentication endpoints, the WebSocket handshake, Swagger UI, <code>/actuator/health info metrics</code> .
Rate limiting	Bucket4j token-bucket, distributed via Redis; a global per-agent limit configured in <code>application.yml</code> (<code>app.rate-limit.global-api.*</code> , overridable with the <code>RATE_LIMIT_GLOBAL_*</code> environment variables).
2FA	Per-user configurable TOTP (authenticator app).
Input security	Bean Validation on all DTOs; additional attachment type/size checks and a sensitive-data scan on text files.
Data isolation	A customer accesses only their own tickets; an agent only the tickets they have claimed / their authorized products; admin and manager are global.

5. Business Processes and jBPM Workflow

The ticket lifecycle is modeled as an executable **BPMN 2.0 process** running on the KIE Server. A ticket is, by its very nature, a long-lived, stateful and event-driven entity. Expressing this behavior with an explicit, visual state machine rather than scattered Java conditionals yields traceability, a durable SLA timer and signal-based transitions.

5.1. Ticket State Machine

BPMN is the authoritative state machine for **all** ticket status changes. A transition advances only when the backend sends the `transition_<TARGET>` signal; the valid transitions are encoded in the BPMN graph itself. An invalid transition is silently ignored by the engine and surfaces as an HTTP `400` in the service layer.

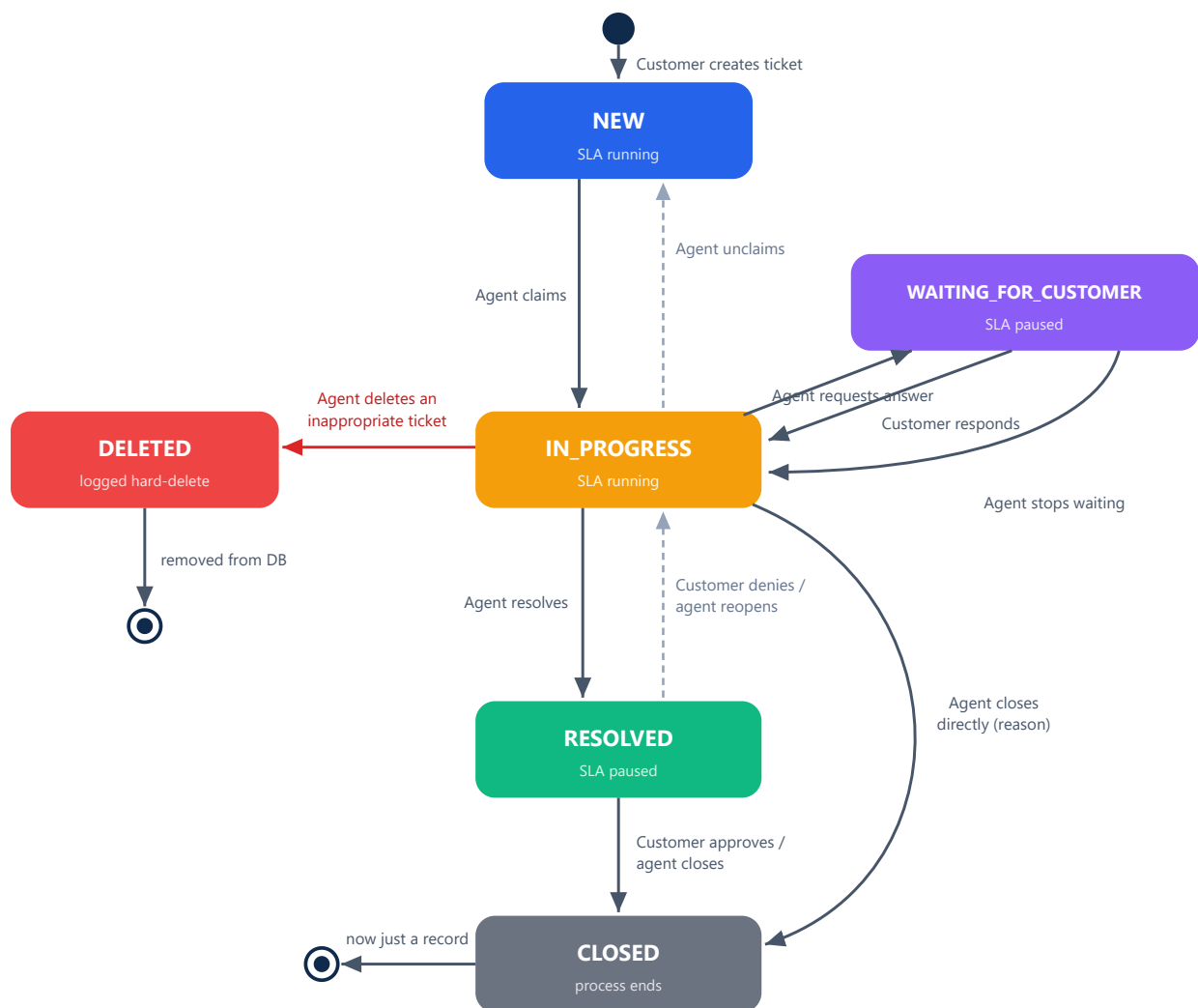


Figure 4. Ticket state machine

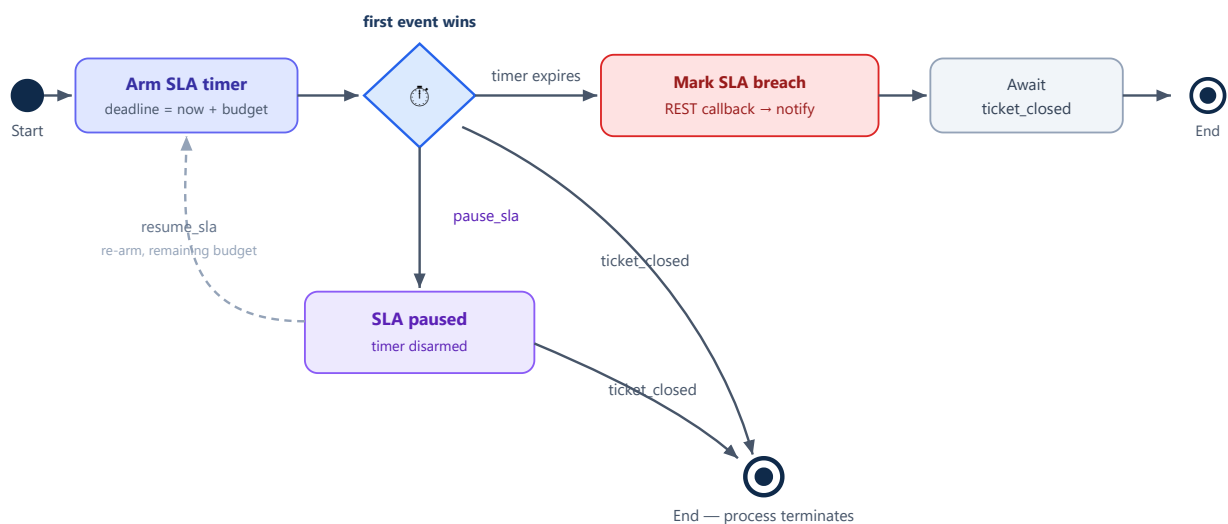
The boxes also show where the SLA clock **runs** (**NEW**, **IN_PROGRESS**) versus **pauses** (**WAITING_FOR_CUSTOMER**, **RESOLVED**). **DELETED** is *not* a lifecycle state: an administrator hard-delete is a logged side-action that removes the ticket permanently from the database, so it has no BPMN status node.

Side-effect transitions also drive BPMN

Not only user-initiated transitions; side-effect transitions driven by *claim/release/assign* also send the same `transition_<STATUS>` signal. Thus a claim performs `NEW → IN_PROGRESS`, and releasing the last claim performs the `IN_PROGRESS → NEW` transition, keeping the BPMN status node consistent with the database.

5.2. BPMN Process Model and Signals

The process runs **two parallel branches** that share the same process instance: (1) the *status branch*, which is the ticket's authoritative state machine, and (2) the event-based *SLA branch* around the SLA timer. The SLA branch reacts to whichever happens first: the SLA timer firing, the `pause_sla` signal, or the `ticket_closed` signal.



Runs in parallel with the status state machine (Figure 4); reaching CLOSED fires a terminate-end event that ends every branch.

Figure 5. jBPM SLA process flow

Communication between the backend and the process is done with named BPMN signals:

Signal	Sender (WorkflowService)	Purpose
<code>transition_<STATUS></code>	<code>requestStatusTransition()</code>	Advances the status node to the corresponding status (NEW / IN_PROGRESS / WAITING_FOR_CUSTOMER / RESOLVED / CLOSED)
<code>pause_sla</code>	<code>pauseSla()</code>	Pauses the SLA counter; adds the elapsed time to <code>slaElapsedMs</code>
<code>resume_sla</code>	<code>resumeSla()</code>	Computes the remaining time and re-arms the timer
<code>ticket_closed</code>	<code>closeTicketWorkflow()</code>	Terminates the process; falls back to <code>abortProcess</code> if the signal errors

5.3. SLA Mechanism

The SLA deadline is computed from the policy by priority when the ticket is created. A timer runs periodically, marking tickets past their deadline as a **breach** and notifying; it also flags those approaching

the warning threshold and notifies the assigned agent.

Priority	Target Resolution	Warning Threshold
CRITICAL	1 hour	at deadline (0 h)
HIGH	4 hours	1 hour before
MEDIUM	24 hours	2 hours before
LOW	72 hours	2 hours before

The SLA counter **pauses** while the ticket is `WAITING_FOR_CUSTOMER` or `RESOLVED`, and restarts when it returns to `IN_PROGRESS`. Only active (counting) time accumulates (`slaElapsedMs`); on resume, the deadline is projected forward by the remaining active budget. This way, customer-caused delays are not counted against the team.

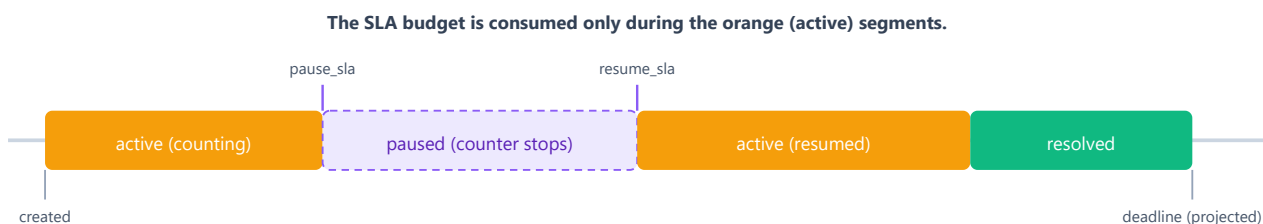


Figure 6. SLA pause / resume timeline

5.4. Resilience: Circuit Breaker and Callback

The KIE Server is treated as a non-critical dependency: ticket operations do not fail just because the workflow engine is unavailable. All KIE calls are wrapped in a Resilience4j **circuit breaker**. The process state itself is persisted to the dedicated `jbpm-db` PostgreSQL instance (not ephemeral in-memory H2), so workflow instances survive a KIE Server restart. A KIE `404` (for example a stale `processInstanceId` whose BPMN instance is gone) is treated as a non-failure: it does not trip the circuit breaker, and the backend accepts the database status transition anyway, so the ticket API stays consistent.

Circuit Breaker Setting	Value
Failure rate threshold	50%
Sliding window size	10 calls
Minimum number of calls	5
Wait duration in open state	30 s
Permitted calls in half-open	3
Slow call threshold	10 s / 50%

KIE → Backend Callback

When the SLA timer fires, the process calls the `POST /api/v1/internal/workflow/callback` endpoint. The request is verified with `X-Internal-Token`; the body contains `ticketId` and `eventType` (`SLA_BREACHED` / `PROCESS_COMPLETED`). `SLA_BREACHED` handling is **idempotent**: if a breach is already flagged, no email is sent again.

6. Database Design and Data Dictionary

6.1. Approach and Standards

- **Relational design and normalization:** The schema conforms to Third Normal Form (3NF) and is enforced with foreign key (FK) constraints.
- **Primary key strategy:** The `users` table uses the Keycloak UUID (VARCHAR(36)); business data (tickets, comments, etc.) uses a `BIGINT` identity for performance.
- **Migration:** All schema changes are managed with versioned Flyway scripts (`V1 ... V37`). Hibernate is in `ddl-auto: validate` mode, so it never alters the schema.
- **Shared schema, isolated history:** `llm-service` shares `ticketdb` but keeps an isolated Flyway history table (`flyway_schema_history_llm`).
- **Audit and integrity:** `created_at` / `updated_at` on critical tables; the ticket's child tables are linked with CASCADE on delete; audit logs are append-only.

The platform spans three logical databases; only the first is designed and owned by the application:

Database	Engine	Owner and contents
<code>ticketdb</code>	PostgreSQL 15	The application schema detailed below. The backend owns the schema and its Flyway history (<code>flyway_schema_history</code>); llm-service shares the very same database with an isolated history table (<code>flyway_schema_history_llm</code>) and persists AI summaries to <code>ticket_ai_summaries</code> . There is no separate "LLM database".
<code>keycloakdb</code>	PostgreSQL 15 (same instance)	Keycloak's own identity schema: users, sessions, clients and realm configuration. Fully managed by Keycloak and opaque to the application.
<code>jbpm-db</code>	PostgreSQL (separate instance)	The KIE Server / jBPM process and history store (process instances, timers, audit). Managed by the workflow engine; keeps in-flight SLA processes durable across restarts.

Sections 6.2–6.4 model the `ticketdb` application schema only; the Keycloak and jBPM schemas are framework-managed and intentionally not detailed here.

6.2. Entity-Relationship Diagram

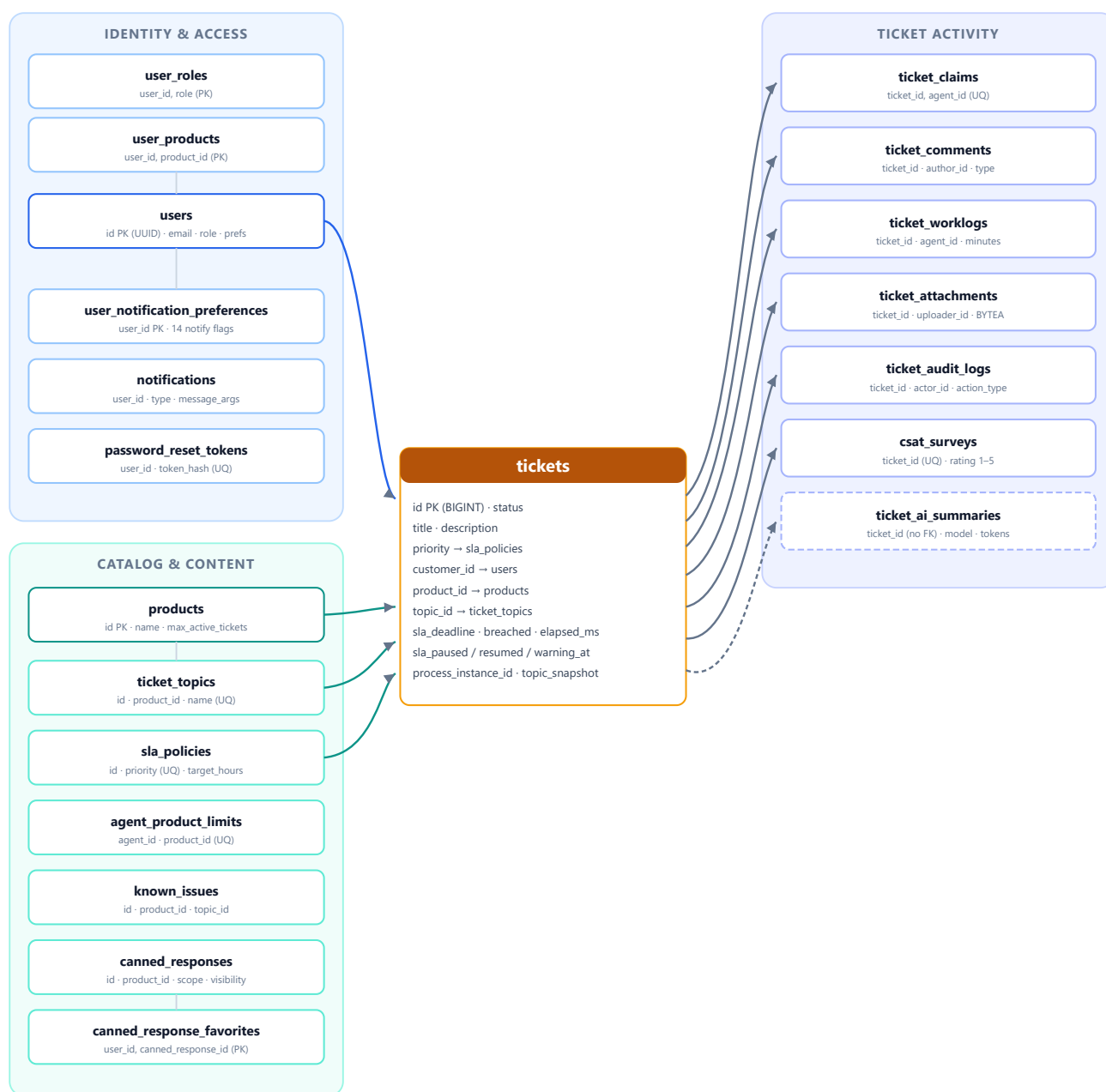


Figure 7. Entity-relationship (ER) diagram (all application tables)

6.3. Table Inventory

As of Flyway V38, the `ticketdb` application schema holds the following tables (the two Flyway history tables aside):

Table	Purpose
<code>users</code>	Users synchronized with Keycloak; language/theme preferences
<code>user_roles</code>	User → role mapping for additive multi-role RBAC (V37)
<code>user_products</code>	User ↔ authorized product many-to-many link
<code>products</code>	Support products/categories

Table	Purpose
<code>ticket_topics</code>	Per-product topic list
<code>sla_policies</code>	SLA targets and warning thresholds by priority
<code>tickets</code>	The main support ticket entity; SLA and workflow fields
<code>ticket_claims</code>	Agent ↔ ticket claims (many-to-many)
<code>ticket_comments</code>	Comments (INTERNAL / EXTERNAL)
<code>ticket_worklogs</code>	Per-ticket effort records (minutes)
<code>ticket_attachments</code>	Ticket attachments (stored as BYTEA)
<code>ticket_audit_logs</code>	Immutable (append-only) operation/status audit log
<code>ticket_ai_summaries</code>	Cached AI-generated ticket summaries (model, token counts, text), written by llm-service
<code>csat_surveys</code>	Satisfaction surveys (1–5 score)
<code>notifications</code>	In-app notifications (i18n key + arguments)
<code>user_notification_preferences</code>	Per-user email/in-app notification preferences
<code>known_issues</code>	Product/topic-based known issues (knowledge base)
<code>canned_responses</code>	Reusable canned-response templates (personal/shared, bilingual TR/EN)
<code>canned_response_favorites</code>	Per-user favourite canned responses (join table)
<code>agent_product_limits</code>	Active ticket limit per agent-product
<code>password_reset_tokens</code>	Single-use, hashed password reset tokens

6.4. Data Dictionary (Selected Tables)

tickets is the system's main table.

Column	Type	Constraint	Description
<code>id</code>	<code>BIGINT</code>	PK	Auto-incrementing ticket number
<code>title</code>	<code>VARCHAR(500)</code>	NOT NULL	Ticket title
<code>description</code>	<code>TEXT</code>	NOT NULL	Problem description
<code>status</code>	<code>VARCHAR(30)</code>	NOT NULL	NEW / IN_PROGRESS / WAITING_FOR_CUSTOMER / RESOLVED / CLOSED
<code>priority</code>	<code>VARCHAR(10)</code>	FK → <code>sla_policies</code>	LOW / MEDIUM / HIGH / CRITICAL
<code>customer_id</code>	<code>VARCHAR(36)</code>	FK → <code>users</code>	The customer who created the ticket
<code>product_id</code>	<code>BIGINT</code>	FK → <code>products</code>	Related product (nullable)
<code>topic_id</code>	<code>BIGINT</code>	FK → <code>ticket_topics</code>	Topic (nullable, ON DELETE RESTRICT)
<code>topic_name_snapshot</code>	<code>VARCHAR(255)</code>	nullable	Denormalized topic name captured on the ticket

Column	Type	Constraint	Description
<code>sla_deadline</code>	<code>TIMESTAMPTZ</code>	nullable	Computed target resolution date
<code>sla_breached</code>	<code>BOOLEAN</code>	DEFAULT false	SLA breach flag
<code>sla_elapsed_ms</code>	<code>BIGINT</code>	DEFAULT 0	Accumulated active SLA time
<code>sla_paused_at</code> / <code>sla_resumed_at</code>	<code>TIMESTAMPTZ</code>	nullable	When the SLA clock was last paused / resumed
<code>sla_warning_sent_at</code>	<code>TIMESTAMPTZ</code>	nullable	When the approaching-deadline warning was sent (prevents duplicates)
<code>process_instance_id</code>	<code>BIGINT</code>	nullable	jBPM process instance identifier
<code>created_at</code> / <code>resolved_at</code> / <code>closed_at</code>	<code>TIMESTAMPTZ</code>	—	Lifecycle timestamps

ticket_comments

Column	Type/Constraint
<code>id</code>	<code>BIGINT</code> PK
<code>ticket_id</code>	FK → tickets (CASCADE)
<code>author_id</code>	FK → users
<code>message</code>	TEXT (≤ 500 chars)
<code>type</code>	EXTERNAL / INTERNAL
<code>created_at</code>	<code>TIMESTAMPTZ</code>

csat_surveys

Column	Type/Constraint
<code>id</code>	<code>BIGINT</code> PK
<code>ticket_id</code>	FK → tickets · UNIQUE
<code>rating</code>	INTEGER (1–5)
<code>comment</code>	TEXT (nullable)
<code>created_at</code>	<code>TIMESTAMPTZ</code>

ticket_ai_summaries (written by the llm-service)

Column	Type/Constraint
<code>id</code>	<code>BIGINT</code> PK
<code>ticket_id</code>	<code>BIGINT</code> (summarized ticket)
<code>model</code>	<code>VARCHAR</code> (100)
<code>prompt/completion_tokens</code>	INTEGER (nullable)
<code>summary</code>	TEXT
<code>created_at</code>	<code>TIMESTAMPTZ</code>

canned_responses

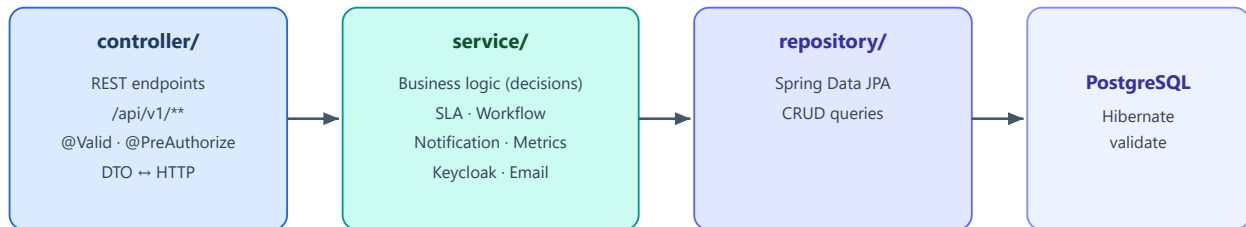
Column	Type/Constraint
<code>id</code>	<code>BIGINT</code> PK
<code>title</code> · <code>shortcut</code>	<code>VARCHAR</code>
<code>content_tr</code> / <code>content_en</code>	TEXT (≥ 1 required)
<code>scope</code>	PERSONAL / SHARED
<code>visibility</code>	EXTERNAL / INTERNAL / BOTH
<code>owner_agent_id</code> · <code>product_id</code>	owner · product (nullable)

Enum value sets: Ticket status (5), priority (LOW/MEDIUM/HIGH/CRITICAL), comment type (INTERNAL/EXTERNAL), canned-response scope (PERSONAL/SHARED) and visibility (EXTERNAL/INTERNAL/BOTH), notification type (TICKET_CREATED, TICKET_ASSIGNED, TICKET_STATUS_CHANGED, COMMENT_ADDED, SLA_WARNING, SLA_BREACHED, TICKET_RESOLVED, GENERAL), roles (customer/agent/lead_agent/admin/manager).

7. Backend Architecture (REST API)

7.1. Layered Structure

The server side is a maintainable and testable layered architecture based on the "Separation of Concerns" principle.



Cross-cutting behaviors: GlobalExceptionHandler · event/ (@Async) · scheduler/ · filter/ · websocket/ · config/

Figure 8. Layered backend architecture

Packages: `controller/`, `service/`, `repository/`, `entity/`, `dto/`, `event/` (`@EventListener` / `@Async`), `scheduler/` (cron), `filter/`, `interceptor/`, `websocket/` (STOMP), `config/` and `exception/`. DTOs form the API boundary; JPA entities are never serialized directly to the client.

7.2. Error Handling and API Contract

Application-wide errors (ticket not found, unauthorized access, validation error, etc.) are caught centrally with `@RestControllerAdvice`. The user receives a standardized JSON error envelope instead of a raw Java stack trace. Bean Validation messages are localized (Turkish/English). The API is documented with springdoc OpenAPI and can be tested live through Swagger UI.

7.3. Critical API Endpoints

All endpoints are served under the `/api/v1/...` prefix. A representative subset is listed below (the required role is enforced with `@PreAuthorize`):

Method	Endpoint	Description	Role
POST	<code>/api/v1/tickets</code>	Creates a new ticket	customer
GET	<code>/api/v1/tickets/pool</code>	Lists unassigned (NEW) tickets	agent, lead_agent
GET	<code>/api/v1/tickets/{id}</code>	Retrieves ticket detail	authenticated
PUT	<code>/api/v1/tickets/{id}/claim</code>	Claims the ticket (NEW→IN_PROGRESS)	agent, lead_agent
PUT	<code>/api/v1/tickets/{id}/assign</code>	Assigns the ticket to another agent	lead_agent, admin
PUT	<code>/api/v1/tickets/{id}/status</code>	Changes status (BPMN validates)	customer*, agent, lead_agent
PUT	<code>/api/v1/tickets/{id}/close</code>	Closes the ticket with a reason	agent, lead_agent
POST	<code>/api/v1/tickets/{id}/comments</code>	Adds a comment (INT/EXT)	authenticated

Method	Endpoint	Description	Role
POST	/api/v1/tickets/{id}/attachments	Uploads a file (≤10MB, checked)	customer, agent, lead_agent
POST	/api/v1/tickets/{id}/worklogs	Adds an effort record	agent, lead_agent
POST	/api/v1/tickets/{id}/csat	Submits the satisfaction survey → closure	customer
GET	/api/v1/tickets/{id}/sla-timer	Retrieves the live SLA countdown	customer, agent, lead_agent, manager
GET	/api/v1/metrics/dashboard-summary	System/team dashboard KPIs (date-range scoped)	manager, lead_agent, admin
GET	/api/v1/metrics/me/customer	The caller's own customer Overview dashboard	authenticated
GET	/api/v1/metrics/me/agent	The caller's own agent My-Performance dashboard	agent, lead_agent
GET	/api/v1/metrics/products/{id}/dashboard	Per-product dashboard (lead is product-scoped)	manager, lead_agent, admin
GET	/api/v1/metrics/users/{id}/agent	Oversight: view a user's agent dashboard	manager, lead_agent, admin
POST	/api/v1/users/sync	Syncs the local user from the JWT	authenticated
POST	/api/v1/users/admin/create	Creates a user in Keycloak	admin
POST	/api/v1/ai/summaries/.../generate	Generates an AI summary (llm-service)	internal token
POST	/api/v1/internal/workflow/callback	JBPM event callback	internal token

* A customer can make only limited status transitions on their own ticket (e.g. [WAITING_FOR_CUSTOMER](#) → [IN_PROGRESS](#) when they reply).

8. Frontend and Mobile Clients

React & Vite Configuration

The interface is built with React 19 and Vite 8. Tailwind CSS 4 is used for styling; icons are provided by `lucide-react` and charts by `Recharts`. API calls are made through an axios interceptor that injects the Keycloak bearer token.

Identity and Session (keycloak-js)

Identity management is done with `keycloak-js`; the JWT is kept in memory and added to the `Authorization: Bearer` header on every request. Session state is held in `AuthContext`, which exposes the role set together with capability helpers (`isAdmin`, `isLeadAgent`, `hasAnyRole`, etc.).

Dynamic Navigation and Route Protection

The sidebar is composed from the **union** of the user's roles. `ProtectedRoute` checks the allowed roles against the role set; a user with no role is redirected to an informative page rather than entering a loop.

Real-time Notifications

The STOMP/WebSocket (`@stomp/stompjs`) connection is established through `AuthContext`; ticket-detail events and notifications are delivered to connected clients in real time.

Internationalization and Theme

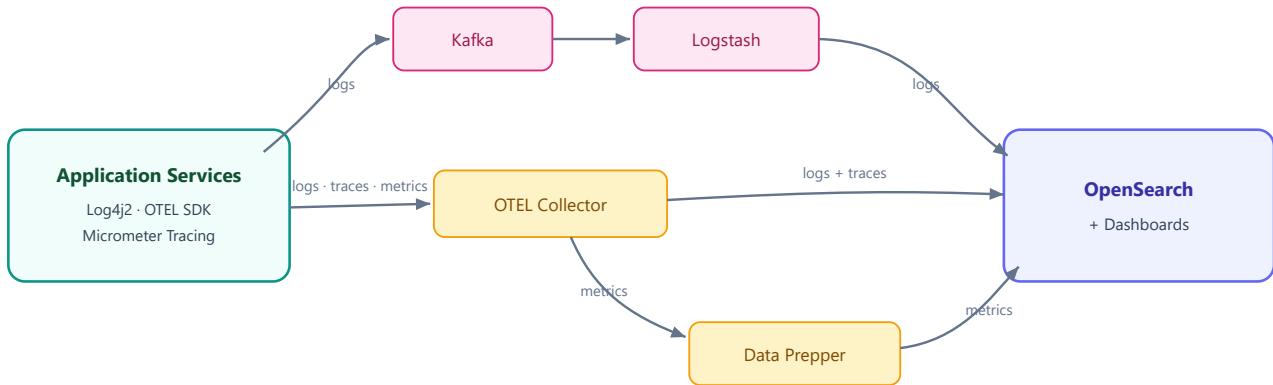
Turkish/English with `i18next`; single-namespace language files. The light/dark theme is shared between the SPA and the custom Keycloak login theme via a domain-scoped cookie.

Mobile App (React Native + Expo)

Functional parity with the web; navigation with React Navigation 7, OIDC PKCE login (`expo-auth-session`), secure token storage (`expo-secure-store`) and the same STOMP-based live notifications. The role model is mirrored one-to-one.

9. Observability

The platform produces **logs, traces and metrics**, all of which converge in OpenSearch. A single observability backend reduces the number of moving parts to operate.



Metrics take the Data Prepper detour because the Collector's OpenSearch exporter cannot write metrics (delta temporality).

Figure 9. Observability data pipeline

- **Logs:** Log4j2 produces structured JSON; shipped both via Kafka → Logstash and via the OpenTelemetry log appender.
- **Traces:** Micrometer Tracing bridges to OpenTelemetry; exported via OTLP (sampling 1.0 in development).
- **Metrics:** Exported with **delta temporality** and routed through Data Prepper, because the collector's OpenSearch exporter does not handle metrics. Delta temporality lets OpenSearch `sum` aggregations work without `rate()`.

10. Resilience and Performance

Mechanism	Purpose
Circuit breaker (Resilience4j)	Isolates jBPM/KIE Server failures from the core API; tolerates a stale process instance (KIE 404) without tripping
Durable workflow state	KIE process state persisted to the <code>jbpm-db</code> PostgreSQL instance, surviving KIE Server restarts
Caching (Caffeine)	Dashboard aggregations are cached with a 5-minute TTL; invalidated on related writes
Rate limiting (Bucket4j + Redis)	Protects the API in general and throttles costly AI calls
Connection pool (HikariCP)	Bounded, tuned database connection pool
Stateless services	Enable horizontal scaling without sticky sessions
Graceful degradation	Workflow/AI outages reduce functionality without crashing the system
Asynchronous processing	Notification and email jobs are kept off the request thread with <code>@Async</code>

The non-functional target is a response time under ~2 seconds for typical operations under normal load.

11. Testing and Quality Assurance

Unit Tests

Backend business logic (the service layer) is tested with JUnit 5 and Mockito; the database and external APIs are mocked. Frontend components are verified with Vitest + Testing Library + jsdom.

Integration Tests

Failsafe runs the `*IT.java` classes; a real PostgreSQL instance is spun up with **Testcontainers** (including Redis). API security and RBAC scenarios are verified end to end.

Coverage and Static Analysis

`mvnw verify` produces the JaCoCo coverage report; static code analysis is done with SonarQube. ESLint is run for the frontend.

Continuous Integration

On every pull request and `main` push, CI runs the backend `verify`, the llm-service build, frontend lint+test+build and Kubernetes manifest validation.

12. Deployment and CI/CD

Two orchestration paths are supported from the same images: **Docker Compose** (`docker-compose.yaml`) for development and demos, and **Kubernetes** (`k8s/` , Kustomize) for near-production scenarios. Kubernetes includes an environment-agnostic `base/` , an `overlays/local` for the local kind cluster and an `overlays/prod` that adds HPA + cert-manager + SealedSecrets.

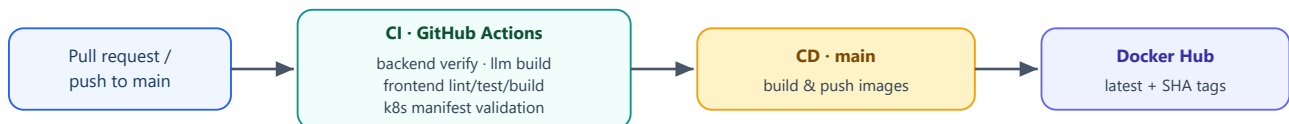


Figure 10. *CI/CD pipeline*

When CI completes successfully on `main` , CD kicks in: Docker images are built and pushed to Docker Hub; for rollback, they are tagged with both `latest` and the commit SHA.

13. Conclusion and Assessment

Within the scope of this document, all the architectural layers required by a modern IT Service & Support Ticket Management System have been detailed. The following table summarizes how the non-functional requirements are met.

Requirement	How It Is Met
Containerization	Each component is an isolated container; Docker Compose and Kubernetes orchestration
Layered backend	Controller → Service → Repository; externalized configuration; standard error format
Persistence	Normalized PostgreSQL schema, JPA/Hibernate, Flyway migrations
Security	Keycloak + LDAP, additive RBAC, 2FA, OAuth2/OIDC; web and mobile
Workflow	A jBPM BPMN process drives ticket status transitions
Observability	OpenTelemetry traces + metrics; unified log/trace/metric dashboards in OpenSearch
Performance	Caching, pooling, rate limiting; sub-~2 s response target
Testing	JUnit 5/Mockito, Testcontainers integration, JaCoCo, SonarQube
DevOps	GitHub Actions CI/CD; image building and publishing
Documentation	This document, the README, the RUNBOOK and the interactive OpenAPI/Swagger reference

13.1. Key Design Decisions

Decision	Rationale
Externalized identity (Keycloak + LDAP)	Standards-based SSO, 2FA and federation; applications stay password-free
Stateless JWT resource server	Horizontal scalability and a clean separation between the identity provider and the API
jBPM for the lifecycle	An explicit, reviewable BPMN process instead of scattered if/else status logic
Circuit breaker around the workflow	The core ticket API stays available even when the KIE Server is down
A single observability backend	Logs/traces/metrics share one store and one query interface
Flyway + <code>validate</code>	The schema is versioned and reviewable; Hibernate cannot change it silently
Shared <code>ticketdb</code> , isolated history	The AI service uses the domain data without cross-service calls